

Milestone 8 Documentation

Milestone 8 focused on implementing the Newton-Raphson method to actually solve the power flow problem. This milestone brought together the mismatch calculations from Milestone 6 and the Jacobian matrix from Milestone 7 into one iterative solver. The main goal was to repeatedly update the unknown bus voltage angles and magnitudes until the mismatch became small enough to meet the tolerance.

Main implementation parts

- Create a PowerFlow class to run the Newton-Raphson solution
- Pull the starting voltage magnitudes and angles from the bus data
- Compute the mismatch vector during each iteration
- Compute the Jacobian matrix
- Solve for the correction vector
- Update bus angles and voltages based on bus type
- Stop when the mismatch is below the tolerance or when the max iteration count is reached

How the code works

In the PowerFlow class, the constructor stores the circuit, settings object, and Jacobian object. It also keeps track of whether the solution converged, how many iterations were used, and the final mismatch vector. This makes it easier to check the result after the solver finishes.

The solve() method starts by creating arrays for the voltage magnitudes and voltage angles using the current values stored in each bus. It then resets the convergence flag and iteration count before entering the Newton-Raphson loop.

Inside the loop, the code first calls the mismatch function to calculate the current mismatch vector. Then it checks if the largest mismatch is already below the tolerance. If it is, the solver stops and marks the solution as converged. If not, it calculates the Jacobian matrix and solves the linear system

$$J\Delta x = f$$

to find the correction vector.

Bus updates

After solving for the correction values, the code updates the unknown variables depending on the bus type:

- **Slack bus**
 - No updates are made
- **PV bus**
 - Only the angle is updated
- **PQ bus**
 - The angle is updated first
 - The voltage magnitude is also updated

What this milestone accomplished

Overall, Milestone 8 was the step where the project started actually solving the system instead of only setting up equations. The solver now uses the mismatch vector and Jacobian together to iteratively move toward the final voltage solution. It also keeps track of convergence, iteration count, and the final mismatch, which helps with checking whether the method worked correctly.